Notes of Reinforcement Learning[1]

Yue Wu

April 5, 2026

add papers!!!

[1] Course ORCS6529 of Columbia University

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Markov Decision Process

$$(\mathcal{S}, \mathcal{A}, R, P)$$

- $\mathcal{S}$: State space
- $\mathcal{A}$: Action space
- R: Reward
- P: Transition prob matrix

If we have finite S states and A actions, then the dimension of Reward is $R \in \mathbb{R}^{S \times A \times S}$, the dimension of Transition prob matrix is $P \in \mathbb{R}^{S \times A \times S}$
At $t = 0, 1, 2, ...$, observe s_t, take action a_t, observe reward r_t and next state s_{t+1}.

About policy

Policy: a mapping from state space to action action $\pi : \mathcal{S} \to \mathcal{A}$

- Markov property and Markovian[?] Due to Markovian property of the MDP, it suffices to consider Markovian policies.
  If an optimal policy exists, there always exists a Markovian policy that is optimal.
- Dimension of a markovian policy

	Deterministic	**Stochastic**
Non-stationary	$H \times S$	$H \times S \times A$
Stationary	$1 \times S$	$1 \times S \times A$

- Horizon and policy type
  - Finite horizon: Non-stationary policy
  - Infinite horizon(with average/discounted reward): such a MDP always has a stationary optimal policy, whenever optimal policy exists[?]

Goal of MDP - Maximize the expectation of total reward[3]

(1) Finite horizon (non-stationary policy)
Find a policy $\pi = (\pi_1, ..., \pi_H), \pi_t \in \Pi$ that maximize the expectation of the total reward of $t = 1, 2, ..., H$.

$$V^\pi(s) = \mathbb{E}\left(\sum_{t=1}^{H} \gamma^{t-1} r_t \mid s_1 = s\right)$$

$$\pi^* \in \arg\max_{\pi \in \Pi^H} V^\pi(s), \quad \forall s \in \mathcal{S}$$

(2) Infinite horizon (stationary policy) [2]
Find a stationary policy $\pi \in \Pi$ that maximize the expectation of the total reward of $t = 1, 2, ...$

$$V^\pi(s) = \mathbb{E}\left(\sum_{t=1}^{\infty} \gamma^{t-1} r_t \,\middle|\, s_1 = s\right), \ \gamma \in (0, 1)$$

$$\pi^* \in \arg\max_{\pi \in \Pi} V^\pi(s), \quad \forall s \in \mathcal{S}$$

[2] Unless stated otherwise, this setting is assumed throughout.

[3] The expectation is taken over trajectories induced by the environment transition dynamics and the policy $\mathbb{E}_{a \sim \pi(s),\, s' \sim P(s,a)}$.

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Bellman equations

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(s),\, s' \sim P(s,a)} \left[R(s, a, s') + \gamma V^{\pi}(s') \right]$$

Optimal Value and Bellman optimality equations

The **optimal value** is defined as

$$V^*(s) = \sup_{\pi} \mathbb{E}\left[\sum_{t=1}^{\infty} \gamma^{t-1} r_t \mid s_1 = s\right]$$

which satisfies the following **Bellman optimality equations**

$$\begin{aligned} V^*(s) &= \max_a \left[R(s,a) + \gamma \sum_{s'} P(s,a,s') V^*(s')\right] \\ &= \max_a \left[R(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)} \left[V^*(s')\right]\right] \end{aligned}$$

- equations that connect $V^*(s)$ for certain s with the values starting from all states $V^*(s)$, $s \in \mathcal{S}$
- self-related(recursive) equations $\rightarrow$ can be solved via iterative algorithms

The **optimal policy** is therefore defined as:

$$\pi^*(s) := \arg\max_a \left[R(s,a) + \gamma \mathbb{E}_{s' \sim P(s,a)} \left[V^*(s')\right]\right]$$

- a policy that attains $V^*(s)$ for all s

How to find the optimal policy/value

When we know the (R, P) of MDP, this is just a simple set of equations to solve. But technically, how?

- Linear programming
- Iterative algorithms
  - Value iteration: Iteratively improve estimate of optimal value(a vector of length S) $[V^*(s),\ s \in \mathcal{S}]$
  - Q-value iteration: Iteratively improve estimate of Q-values(a tabular of size $S \times A$) $[Q^*(s, a),\ s \in \mathcal{S},\ a \in \mathcal{A}]$
  - Policy iteration: Iteratively improve estimate of optimal policy(a vector of length S) $[\pi^*(s),\ s \in \mathcal{S}]$)

Q-value and Bellman equations

Q-values qualifies the first action under a certain policy for following steps. The introduction of Q-value make it possible for us to get the optimal policy by optimal value without knowing transition model P by $\pi(s) = \arg\max_a Q(s, a)$.

Q^*: the expected utility on taking action a in state s, and thereafter acting optimally

$$\begin{aligned} Q^*(s, a) &:= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') V^*(s') \\ &= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') \left(\max_{a'} Q^*(s', a') \right) \end{aligned}$$

Q^π: the expected utility on taking action a in state s, and thereafter playing policy π

$$\begin{aligned} Q^\pi(s, a) &:= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') V^\pi(s') \\ &= R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') \mathbb{E}_{a' \sim \pi(s)} (Q^\pi(s', a')) \end{aligned}$$

Q-value iteration

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. In every iteration $k = 0, 1, 2, \ldots$, improve the Q-value as

$$Q^k(s, a) = R(s, a) + \gamma \sum_{s'} P(s, a, s') \left(\max_{a'} Q^{k-1}(s', a') \right) \quad \forall s, a$$

3. Stop if $\|Q^k - Q^{k-1}\|_\infty$ is small
4. Output policy π_k, where

$$\pi_k(s) = \arg\max_a Q^k(s, a)$$

Policy iteration using Q-values

Iteratively improve estimate of optimal policy.

1. Initialize policy π^0.
2. In every iteration $k = 0, 1, 2, \ldots,$

- (Policy evaluation) Compute $Q^{\pi^k}(s, a), \ \forall s, a$
- (Greedy Policy improvement) Compute new policy

$$\pi^{k+1}(s) := \arg\max_a Q^{\pi^k}(s, a), \forall s$$

3. Stop when $\pi^{k+1} = \pi^k$ or not much change in policy.

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Reinforcement Learning

- Learning from observations

  When we do not know the (R, P) of the MDP of interest, we can interact with the environment to estimate them.

$$R(s, a) = \mathbb{E}(r_t \mid S_t = s, a_t = a)$$

$$P(s, a, s') = P(S_{t+1} = s' \mid S_t = s, a_t = a)$$

- Question
  - How to solve the best policy(implement Q-value iteration and policy iteration) using samples?
  - How to interact with the environment to generate useful samples?

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Tabular Q-learning

Q-learning is a **sample-based version of Q-value iteration**.

- recall the update step in Q-learning

$$Q^k(s,a) = R(s,a) + \gamma \sum_{s'} P(s,a,s') \left(\max_{a'} Q^{k-1}(s',a') \right) \quad \forall s, a$$

- A naive method
  For $k = 1, 2, ...$, observe s_k, take a random action a_k(uniformly choose from the action space), observe reward r_k and next state s_{k+1}, **improve the one-point Q-value at** (s_k, a_k) as

$$Q^k(s_k, a_k) = r_t + \gamma \left(\max_{a'} Q^{k-1}(s_{k+1}, a') \right)$$

  - unbiased but large variance

Tabular Q-learning

Two ways to make the algorithm more steady and efficient

- Slow down the update(e.g. with learning rate α_k)[4]

$$Q^k(s_k, a_k) = (1-\alpha_k)Q^{k-1}(s_k, a_k)+\alpha_k \left(r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a'))\right)$$

- Choose action a_k smartly(e.g. ϵ-greedy)

$$a_t = \arg\max_a Q^{t-1}(s_t, a), \text{ w.p. } 1-\epsilon$$

[4] The update can also be expressed in a Temporal difference(TD) sense

$$Q^k(s_k, a_k) \leftarrow Q^{k-1}(s_k, a_k) + \alpha_k \delta$$

where TD is $\delta = r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a')) - Q^{k-1}(s_k, a_k)$

Tabular Q-learning Algorithm

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. In every iteration $k = 1, 2, \ldots$
   - observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{k-1}(s_k, a), \text{ w.p. } 1 - \epsilon$$

   - observe reward r_k and next state s_{k+1}
   - improve the one-point Q-value at (s_k, a_k) as

$$Q^k(s_k, a_k) \leftarrow (1-\alpha_k)Q^{k-1}(s_k, a_k) + \alpha_k \left(r_k + \gamma(\max_{a'} Q^{k-1}(s_{k+1}, a')) \right)$$

3. Stop if $\|Q^k - Q^{k-1}\|_\infty$ is small
4. Output policy π_k, where

$$\pi_k(s) = \arg\max_a Q^k(s, a)$$

Large-scale Q-learning

If we are facing large state space, the Q-tabular would become huge and hard to update. How to reduce the dimension? By **parameterizing tabular Q-values into a model**

$$Q_\theta(s, a) = f_\theta(x(s, a))$$

where the vector $x(s, a)$(feature embedding) can have less dimension, and $f_\theta(\cdot)$ can be any function of the vector(e.g. Linear regression or DNN). So **updating Q tabular** $\rightarrow$ **updating** θ **by gradient of loss**.

$$Q^k(s_k, a_k) \leftarrow (1 - \alpha_k) Q^{k-1}(s_k, a_k) + \alpha_k \Big(r_t + \gamma (\max_{a'} Q^{k-1}(s_{k+1}, a') \Big)$$

$$L(\theta_{k-1}) = L(Q_{\theta_{k-1}}(s_k, a_k), r_k + \gamma (\max_{a'} Q_{\theta_{k-1}}(s_{k+1}, a'))$$

So the parameter is updated as

$$\theta_k \leftarrow \theta_{k-1} - \alpha_k \nabla_\theta L(\theta_{k-1})$$

Large-scale Q-learning

When loss is defined by $L(x, y) = \frac{1}{2}(x - y)^2$, the parameter is updated as

$$\theta_k \leftarrow \theta_{k-1} + \alpha_k \delta_k \nabla_\theta Q_{\theta_{k-1}}(s, a)$$

where

- TD target: $y = r_k + \gamma \max_{a'} Q_{\theta_{k-1}}(s_{k+1}, a')$
- TD error: $\delta_k = r_k + \gamma \max_{a'} Q_{\theta_{k-1}}(s_{k+1}, a') - Q_{\theta_{k-1}}(s_k, a_k)$
- learning rate/step size: α_k

Large-scale Q-learning Algorithm

Deep Q-learning algorithm (DQN)

Iteratively improve estimate of parameter θ.

1. Initialize θ^0
2. In every iteration $k = 1, 2, \ldots$
   - observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{\theta_{k-1}}(s_t, a), \text{ w.p. } 1 - \epsilon$$

   - observe reward r_k and next state s_{k+1}
   - improve the parameter as

$$\theta_k \leftarrow \theta_{k-1} - \alpha_k \nabla_\theta L(\theta_{k-1})$$

3. Stop if $\|\theta^k - \theta^{k-1}\|_\infty$(or gradient, etc) is small
4. Output policy θ_k, where

$$\pi_k(s) = \arg\max_a Q^{\theta_k}(s, a)$$

Large-scale Q-learning Algorithm

implementation tips

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Tabular TD-learning

Recall the policy iteration with Q-values, there is a step called policy evaluation, where we **compute the Q-values for each state and action for a given policy** $Q^\pi(s, a)$, $\forall s, a$. TD-learning is a **sample-based approximate version of policy evaluation**.

Computing Q-values of a fixed policy π:

- Monte Carlo method
- Q-value-iteration: Based on Bellman equation (DP)

$$Q^\pi(s, a) = R(s, a) + \gamma \sum_{s' \in S} P(s, a, s') \mathbb{E}_{a' \sim \pi(s)}(Q^\pi(s', a'))$$

Tabular TD-learning algorithm for learning Q-values of policy π

A subroutine of policy iteration using Q-values

Iteratively improve estimate of Q-values.

1. Initialize $Q^0 \in \mathbb{R}^{S \times A}$
2. In every iteration $k = 1, 2, \ldots$
   - observe s_k, take action a_k using ϵ-greedy policy

$$a_k = \arg\max_a Q^{k-1}(s_k, a), \text{ w.p. } 1 - \epsilon$$

   - observe reward r_k and next state s_{k+1}
   - improve the one-point Q-value at (s_k, a_k) as

$$Q^k(s_k, a_k) \leftarrow (1-\alpha_k)Q^{k-1}(s_k, a_k) + \alpha_k \left(r_k + \gamma(\mathbb{E}_{a' \sim \pi(s_{t+1})} Q^{k-1}(s_{k+1}, a'))\right)$$

3. Stop if $\|Q^k - Q^{k-1}\|_\infty$ is small

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Policy gradient methods

Q-learning and TD-learning:

- Q-value-based method, not targeting policy directly
- deterministic policy (argmaxed by Q-values)
- Limitation: an arbitrarily small change in the estimated value can cause the action to be or not to be selected, hard to establish convergence assurances

Policy gradient methods:

- **stochastic policy**
- use **function form of policy** for **directly finding optimal policy**

Policy gradient methods

Parameterizing the policy space as

$$\pi_\theta : S \times A \to R, \ \sum_a \pi_\theta(s,a) = 1$$

- Examples
  - Discrete action space.

$$\pi_\theta(s,a) = \frac{e^{f_{\theta_a}(x_s)}}{\sum_{a' \in A} e^{f_{\theta_{a'}}(x_s)}}$$

  - Continuous unrestricted action space

$$\pi_\theta(s) = \mathcal{N}(f_\theta(s), \sigma^2)$$

Therefore, goal of finding optimal policy is equivalent to finding the optimal parameter that,

$$\theta^* = \arg\max_\theta V^{\pi_\theta}(s)$$

A natural question: Can we compute/estimate the gradient?

$$\theta_{k+1} \leftarrow \theta_k + \alpha_k \nabla_\theta V^{\pi_\theta}(s)$$

Policy gradient methods - Finite horizon

Goal:

$$V^{\pi}(s_1) = \mathbb{E}\left(\sum_{t=1}^{H} \gamma^{t-1} r_t \mid s_1\right)$$

Theorem: for a policy π_θ

$$\nabla_\theta V^{\pi_\theta}(s_1) = \mathbb{E}_\tau\left[R(\tau)\sum_{t=1}^{H} \nabla_\theta \log(\pi_\theta(s_t, a_t)) \mid s_1\right]$$

where $R(\tau)$ can be estimated, $\nabla_\theta \log(\pi_\theta(s, a))$ can be computed.

Policy gradient methods - Finite horizon

Algorithm(Vanilla Policy gradient algorithm)

1. Initialize θ^0
2. In every iteration $k = 1, 2, \ldots$
   - Execute current policy π_θ to obtain several sample trajectories $\tau_i, i = 1, ..., m$
   - observe rewards and calculate the empirical total discounted reward $\hat{R}(\tau_i)$ of each trajectory $\hat{R}(\tau) = r_1 + \gamma r_2 + ...\gamma^{H+1} r_H$
   - Use these sample trajectories and chosen baseline to compute an unbiased gradient estimator using Policy gradient theorem

$$\hat{\mathbf{g}}_k = \frac{1}{m} \sum_{i=1}^{m} \hat{R}(\tau_i) \sum_{t=1}^{H} \nabla_\theta \log\big(\pi_\theta(s_t^i, a_t^i)\big)$$

   - Update:

$$\theta_{k+1} \leftarrow \theta_k + \alpha \hat{\mathbf{g}}_k.$$

Policy gradient methods - Infinite horizon

Goal:

$$V^{\pi_\theta}(s_1) = \lim_{T\to\infty} \mathbb{E}\left(\sum_{t=1}^{T} \gamma^{t-1} r_t \mid s_1\right)$$
$$= \sum_s d^{\pi_\theta}(s \mid s_1) \sum_a \pi_\theta(s,a) R(s,a)$$

where $d^{\pi_\theta}(s \mid s_1)$ is the total discounted prob of being in s under π

$$d^{\pi_\theta}(s) = \lim_{T\to\infty} \sum_{t=1}^{T} \gamma^{t-1} P(s_t = s \mid s_1, \pi)$$

Theorem: for a policy π_θ

$$\nabla_\theta V^{\pi_\theta}(s_1) = \sum_s d^{\pi_\theta}(s \mid s_1) \sum_a Q^{\pi_\theta}(s,a) \nabla_\theta \pi_\theta(s,a)$$
$$= \frac{1}{1-\gamma} \mathbb{E}_{s\sim(1-\gamma)d^{\pi_\theta}, a\sim\pi_\theta(s)} \left[Q^{\pi_\theta}(s,a) \nabla_\theta \log(\pi_\theta(s,a))\right]$$

where $Q^{\pi_\theta}(s,a)$ can be estimated, $\nabla_\theta \log(\pi_\theta(s,a))$ can be computed.

$$\nabla_\theta V(\theta) = \mathbb{E}_{s\sim d^\pi, a\sim\pi} \left[A^\pi(s,a) \nabla_\theta \log \pi_\theta(a|s)\right]$$

Policy gradient methods - Infinite horizon

Vanilla Policy Gradient Algorithm

1. Initialize policy parameter θ^0.
2. In each iteration $k = 1, 2, \ldots$:
   - Execute current policy π_θ to obtain several sample trajectories $\tau_i, i = 1, ..., m$
   - For any given sample trajectory i, use observed rewards to compute an unbiased estimate of $Q(s_t, a_t)$

$$\hat{Q}_t^i := \sum_{t'=t}^{T-1} \gamma^{t'-t} r_{t'}^i.$$

   - Use $\hat{Q}_t^i$ to compute a gradient estimator $\hat{g}_k$ using Policy Gradient Theorem.

$$\hat{\mathbf{g}} = \frac{1}{m} \sum_{i=1}^{m} \sum_{t=1}^{T} \gamma^{t-1} \hat{Q}_t^i \nabla_\theta \log\big(\pi_\theta(s_t^i, a_t^i)\big)$$

Update:

$$\theta_{k+1} \leftarrow \theta_k + \alpha \hat{g}_k.$$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Problem of Policy gradient methods

- Problems of policy gradient methods
  - iterate by gradient does not mean improvement of policy (examples show that the improvement in policy in every step can either be very small, or may even be negative)
  - hard to quantify the number of steps required for this algorithm to converge
- Can we design an algorithm that is guaranteed to improve some performance measure at every step?

Advantage function and Performance difference lemma

- Advantage function

  The advantage of action a in state s

$$A^{\pi}(s,a) = Q^{\pi}(s,a) - V^{\pi}(s)$$

- Performance difference lemma

  For two policy $\tilde{\pi}, \pi$ start at same state s_1

$$\begin{aligned} V^{\tilde{\pi}}(s_1) - V^{\pi}(s_1) &= \sum_s d^{\tilde{\pi}}(s) \sum_a \tilde{\pi}(s,a) A^{\pi}(s,a) \\ &= \frac{1}{(1-\gamma)} \mathbb{E}_{s \sim (1-\gamma) d^{\tilde{\pi}}, a | s \sim \tilde{\pi}} [A^{\pi}(s,a)] \end{aligned}$$

Policy improvement theorem

For any two policies π,π', let π^{new} be

$$\pi_\alpha^{\text{new}}(s,a) = (1-\alpha)\pi(s,a) + \alpha\pi'(s,a)$$

The following theorem **quantifies the least change of value of the new policy with the original policy** with step size α and policy π'.

$$V^{\pi_\alpha^{\text{new}}}(s_1) - V^{\pi}(s_1) \geq \frac{\alpha}{1-\gamma} A_\pi(\pi') - \frac{\alpha^2}{(1-\gamma)^2} 2\gamma A_\pi^{\max}$$

where

$$A_\pi(\pi') = \mathbb{E}_{s\sim(1-\gamma)d^\pi}\left[\sum_a \pi'(s,a)A^\pi(s,a)\right]$$

$$A_\pi^{\max} = \max_s \left|\sum_a \pi'(s,a)A^\pi(s,a)\right|$$

Conservative Greedy policy improvement algorithm

Policy improvement theorem helps us **design an algorithm that always have a positive improvement in policy value by choosing α and π'.**

- For α, we can select a step size to always have a positive improvement as long as $A_\pi(\pi') > 0$. Let R be an upper bound on rewards, so that $A_\pi(\pi') \leq A_\pi^{\max} \leq \frac{R}{1-\gamma}$. Then, setting

$$\alpha = \frac{A_\pi(\pi')(1-\gamma)^3}{4R}$$

- For π', intuitively we can choose a policy that is greedy

$$\pi'(s) = \arg\max_a A^\pi(s, a)$$

which guaranteed that $A_\pi(\pi') =$ is maximized

Conservative Greedy policy improvement algorithm

algorithm:
Q: policy improvement and global monotone?

Parameterized Conservative Greedy policy improvement algorithm

$$\theta^*_{k+1} = \arg\max_{\theta} \mathbb{E}_{s\sim(1-\gamma)d^{\pi_{\theta_k}}} \left[\sum_a \pi_\theta(s,a)\, A^{\pi_{\theta_k}}(s,a) \right]$$

Outline

Introduction
- MDP
- Solve an MDP when MDP is known

RL: Solve an MDP when MDP is unknown
- Reinforcement Learning
- Q-learning
- TD-learning
- Policy gradient methods
- Conservative Greedy policy improvement algorithm
- Trust Region Policy Optimization (TRPO)

Bandits

Trust Region Policy Optimization (TRPO)

- Conservative greedy policy improvement algorithm
  - every iteration adds a new policy, costing a lot of space.
- Trust Region Policy Optimization (TRPO)
  - just need to maintain and update the policy parameter
  - same the idea of a conservative greedy policy by **bounding the difference between old and new policy**
  - theoretically good and technically good

Trust Region Policy Optimization (TRPO)

In each iteration $k+1$:

$$\theta_{k+1} = \begin{array}{rl} \arg\max_\theta & \mathbb{E}_{s\sim\pi_{\theta_k}}\left[\sum_a \pi_\theta(s,a) A^{\pi_{\theta_k}}(s,a)\right] \\ \text{s.t.} & \mathbb{E}_{s\sim\pi_{\theta_k}}\left[D_{\mathrm{KL}}(\pi_\theta(\cdot|s), \pi_{\theta_k}(\cdot|s))\right] \leq \delta \end{array}$$

Proximal Policy Optimization (PPO)

PPO replaces the trust region constrained by clipped ratio in the objective and solve the unconstrained problem.

references